

Lecture 9

Moving Beyond Linearity

9.1

Contents

1	Moving Beyond Linearity	1
2	Polynomial Regression	1
3	Step Functions	2
4	Piecewise Polynomials	4
5	Linear Splines	5
6	Cubic Splines	6
7	Natural Cubic Splines	7
8	Some Remarks on Degrees of Freedom	8
9	Knot Placement	9
10	Smoothing Splines	9
11	Local Regression	11
12	Generalized Additive Models	12

Figures in this presentation are taken from 'An Introduction to Statistical Learning, with applications in R' (Springer, 2013) with permission from the authors: G. James, D. Witten, T. Hastie and R. Tibshirani.

Most of the material is extracted from the slides of the MOOC course by Hastie and Tibshirani, available at <https://www.r-bloggers.com/in-depth-introduction-to-machine-learning-in-15-hours-of-expert-videos/>.

9.2

1 Moving Beyond Linearity

The truth is never linear!

Or almost never!

But often the linearity assumption is good enough.

When it's not we can have recourse to:

- polynomials,

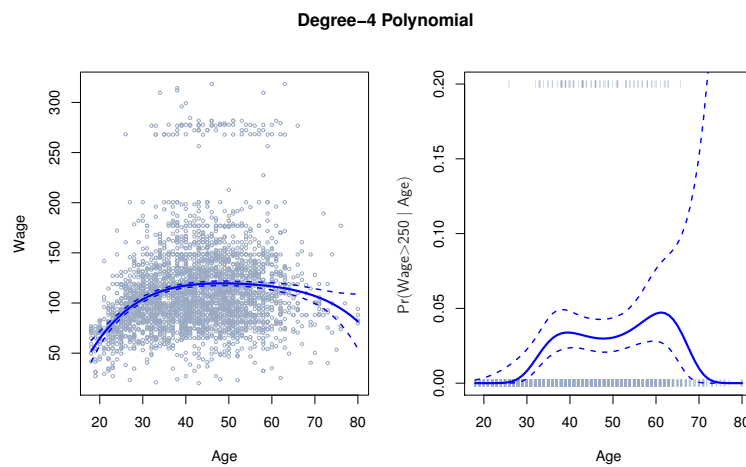
- step functions,
- splines,
- local regression, and
- generalized additive models

These methods offer a lot of flexibility, without losing the ease and interpretability of linear models.

9.3

2 Polynomial Regression

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \dots + \beta_d x_i^d + \varepsilon_i$$



9.4

Details

- Create new variables $X_1 = X$, $X_2 = X^2$, etc. and then treat as multiple linear regression.
- We are not really interested in the coefficients; more interested in the fitted function values at any value x_0 :

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4.$$

- Since $\hat{f}(x_0)$ is a linear function of the $\hat{\beta}_i$, we can get a simple expression for the variance of $\hat{f}(x_0)$ at any value x_0 . In the figure we have computed the fit and pointwise standard errors on a grid of values for x_0 . Dotted curves correspond to $\hat{f}(x_0) \pm 2 \cdot se[\hat{f}(x_0)]$.
- We either fix the degree d of the polynomial at some reasonably low value, else use cross-validation to choose d .

9.5

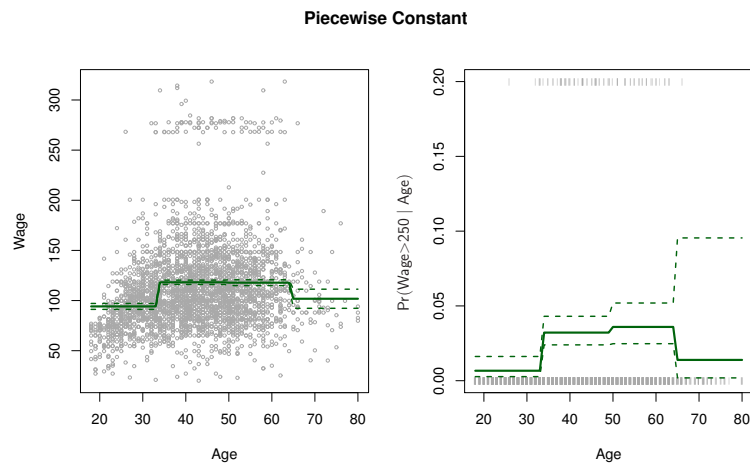
- The second figure refers to a logistic regression model.
- The method can be applied separately on several variables (see GAMs later).
- Caveat: polynomials have notorious tail behavior – very bad for extrapolation.
- The model can be fitted with R by using `y ~ poly(x, degree = 3)`

9.6

3 Step Functions

Another way of creating transformations of a variable – cut the continuous variable into distinct regions.

$$C_1(X) = I(X < 35), C_2(X) = I(35 \leq X < 50), C_3(X) = I(50 \leq X < 65), C_4(X) = I(X \geq 65)$$



9.7

- Easy to work with. Creates a series of dummy variables representing each group.
- Useful way of creating interactions that are easy to interpret. For example, interaction effect of Year and Age:

$$I(\text{Year} < 2005) \cdot \text{Age}, \quad I(\text{Year} \geq 2005) \cdot \text{Age}$$

would allow for different linear functions in each age category.

- In R: `I(year < 2005)` and `cut(age, c(18, 25, 40, 65, 90))`.
- Choice of cutpoints or *knots* can be problematic. For creating nonlinearities, smoother alternatives such as *splines* are available.

9.8

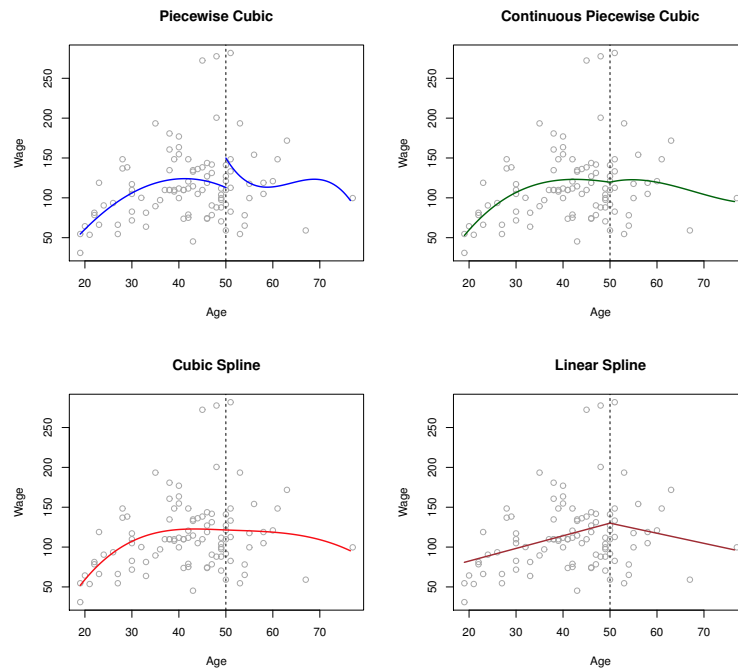
4 Piecewise Polynomials

- Instead of a single polynomial in X over its whole domain, we can rather use different polynomials in regions defined by knots. E.g. (see figure)

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \varepsilon_i & \text{if } x_i < c, \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \varepsilon_i & \text{if } x_i \geq c. \end{cases}$$

- Better to add constraints to the polynomials, e.g. continuity.
- *Splines* functions have the "maximum" amount of continuity.

9.9



9.10

5 Linear Splines

A linear spline with knots at $\xi_k, k = 1, \dots, K$ is a piecewise linear polynomial continuous at each knot.

We can represent this model as

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \dots + \beta_{K+1} b_{K+1}(x_i) + \varepsilon_i,$$

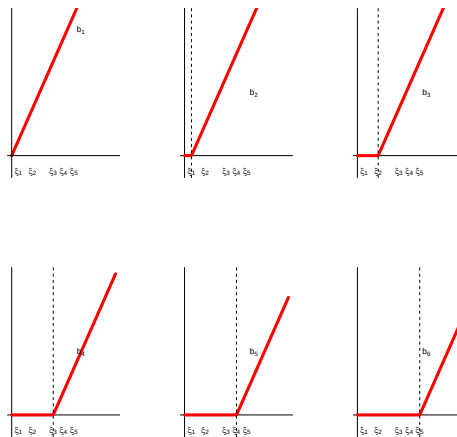
where the $b_1, b_2, \dots, b_{K+1}$ are *basis functions*.

$$\begin{aligned} b_1(x_i) &= x_i \\ b_{k+1}(x_i) &= (x_i - \xi_k)_+, \quad k = 1, \dots, K \end{aligned}$$

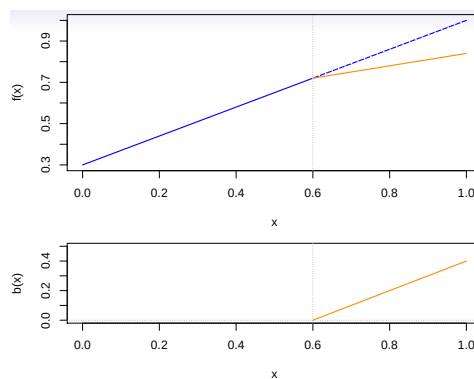
Here the $(\cdot)_+$ means *positive part*; i.e.

$$(x_i - \xi_k)_+ = \begin{cases} x_i - \xi_k & \text{if } x_i > \xi_k \\ 0 & \text{otherwise} \end{cases}$$

9.11



9.12



9.13

6 Cubic Splines

A cubic spline with knots at ξ_k , $k = 1, \dots, K$ is a piecewise cubic polynomial with continuous derivatives up to order 2 at each knot.

Again we can represent this model with truncated power basis functions

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \dots + \beta_{K+3} b_{K+3}(x_i) + \varepsilon_i.$$

$$b_1(x_i) = x_i$$

$$b_2(x_i) = x_i^2$$

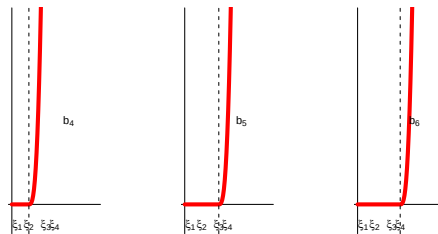
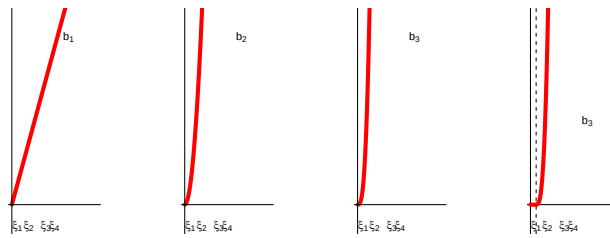
$$b_3(x_i) = x_i^3$$

$$b_{k+3}(x_i) = (x_i - \xi_k)_+^3, \quad k = 1, \dots, K$$

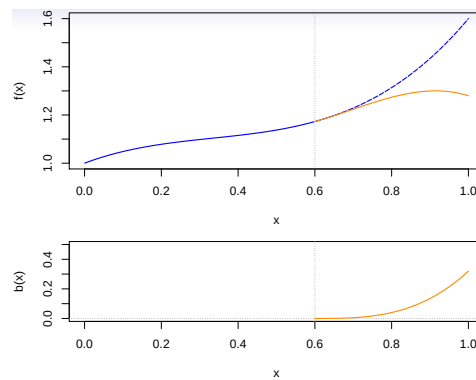
where $(\cdot)_+$ is positive part

$$(x_i - \xi_k)_+^3 = \begin{cases} (x_i - \xi_k)^3 & \text{if } x_i > \xi_k \\ 0 & \text{otherwise} \end{cases}$$

9.14



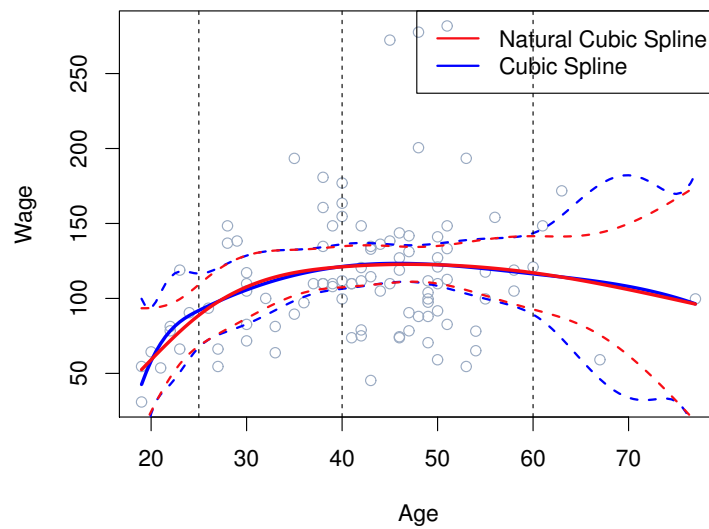
9.15



9.16

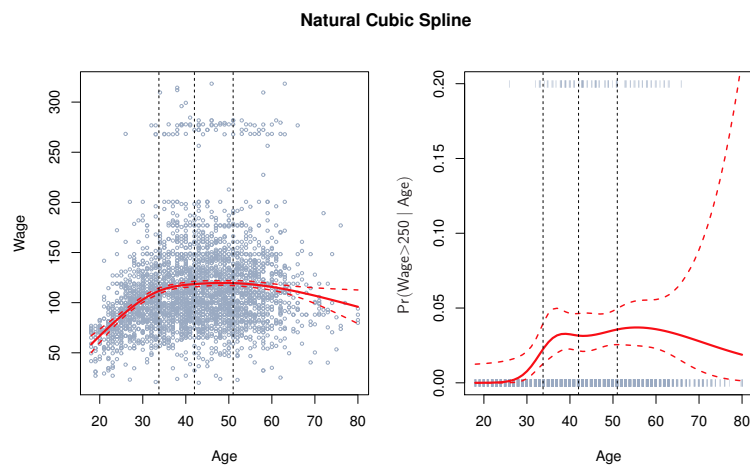
7 Natural Cubic Splines

A natural cubic spline extrapolates linearly beyond the boundary knots. It is characterized by the constraint that polynomials in the extreme intervals have zero second derivatives. This adds $4 = 2 \times 2$ extra constraints, and allows us to put more internal knots for the same degrees of freedom as a regular cubic spline.



9.17

Fitting splines in R is easy: `bs(x, ...)` for any degree splines, and `ns(x, ...)` for natural cubic splines, in package `splines`.



9.18

8 Some Remarks on Degrees of Freedom

With Piecewise Polynomials we have in the 2 knots situation:

1 st polynomial	 ξ_1	2 nd polynomial	 ξ_2	3 rd polynomial
$\beta_{01} + \beta_{11}x + \beta_{21}x^2 + \beta_{31}x^3$		$\beta_{02} + \beta_{12}x + \beta_{22}x^2 + \beta_{32}x^3$		$\beta_{03} + \beta_{13}x + \beta_{23}x^2 + \beta_{33}x^3$
4 parameters		4 parameters		4 parameters

Therefore, with K knots we have to estimate:

$$4(K + 1)$$

parameters (degrees of freedom).

9.19

Cubic splines require, at each node, continuity of:

1. the function,
2. its first derivative, and
3. its second derivative,

so defining $3K$ constraints. Therefore, there remain

$$4(K + 1) - 3K = 4 + K$$

degrees of freedom.

9.20

Natural cubic splines require that the 'external' functions are linear.

1 st polynomial	 ξ_1	2 nd polynomial	 ξ_2	3 rd polynomial
$\beta_{01} + \beta_{11}x$		$\beta_{02} + \beta_{12}x + \beta_{22}x^2 + \beta_{32}x^3$		$\beta_{03} + \beta_{13}x$
4 - 2 parameters		4 parameters		4 - 2 parameters

Therefore, natural splines require 4 more constraints (coefficients of linear and quadratic terms of external cubic functions are null), and there remain

$$4 + K - 4 = K$$

degrees of freedom.

9.21

This is equivalent to say that natural cubic splines have two additional knots ξ_0 and ξ_K

 ξ_0	1 st polynomial	 ξ_1	2 nd polynomial	 ξ_2	3 rd polynomial	 ξ_K
-------------	----------------------------	-------------	----------------------------	-------------	----------------------------	-------------

and external polynomials satisfy the following 4 constraints at the external knots:

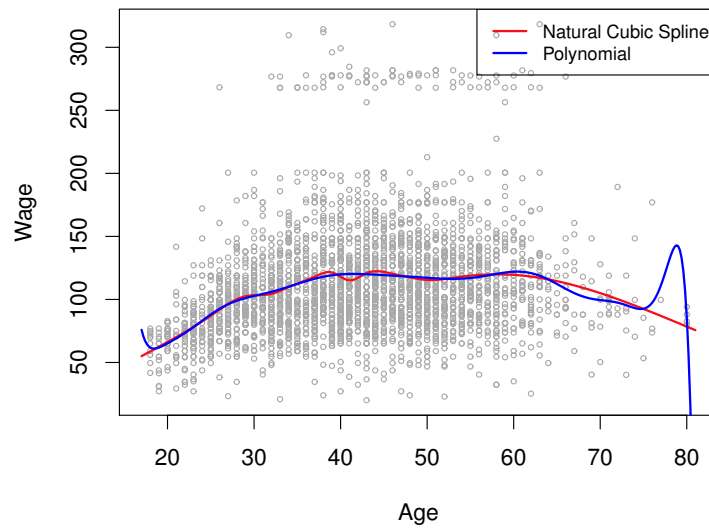
- $\hat{y}_i(\xi_0) = E(y_i|\xi_0)$ and $\hat{y}_i(\xi_K) = E(y_i|\xi_K)$
- the second derivative of the external functions is null.

9.22

9 Knot Placement

- One strategy is to decide K , the number of knots, and then place them at appropriate quantiles of the observed X .
- A cubic spline with K knots has $K + 4$ parameters or degrees of freedom.
- A natural spline with K knots has K degrees of freedom.

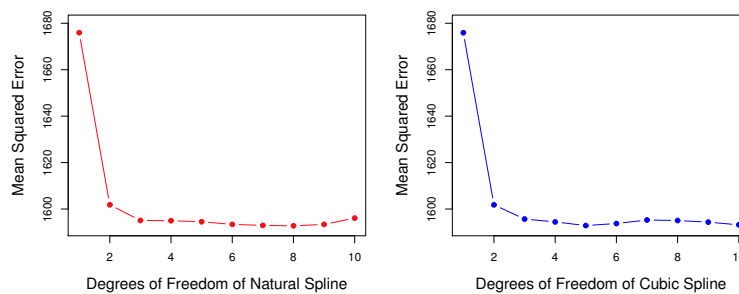
Comparison of a degree-14 polynomial and a natural cubic spline, each with 15df.



```
ns(age, df=14) poly(age, deg=14)
```

9.23

Degrees of freedom can be selected with cross validation. The following graphs show that the linear model, corresponding to 1 df (0 knots), is not adequate according to the estimate of test MSE.



9.24

10 Smoothing Splines

Consider this criterion for fitting a smooth function $g(x)$ to some data:

$$\arg \min_{g \in \mathcal{F}} \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt$$

- The first term is RSS, and tries to make $g(x)$ match the data at each x_i .
- The second term is a *roughness penalty* and controls how wiggly $g(x)$ is. It is modulated by the *tuning parameter* $\lambda \geq 0$.
 - The smaller λ , the more wiggly the function, eventually interpolating y_i when $\lambda = 0$.
 - As $\lambda \rightarrow \infty$, the function $g(x)$ becomes linear.

9.25

The solution is a natural cubic spline, with a knot at every unique value of x_i . The roughness penalty still controls the roughness via λ .

Some details

- Smoothing splines avoid the knot-selection issue, leaving a single tuning parameter λ to be chosen.
- The algorithmic details are too complex to describe here.
In R, the function `smooth.spline()` will fit a smoothing spline.
- The vector of n fitted values can be written as $\hat{g}_\lambda = \mathbf{S}_\lambda \mathbf{y}$, where $\mathbf{S}$ is a $n \times n$ matrix (determined by the x_i and λ).
- The *effective degrees of freedom* are given by

$$df_\lambda = \sum_{i=1}^n \{\mathbf{S}_\lambda\}_{ii}.$$

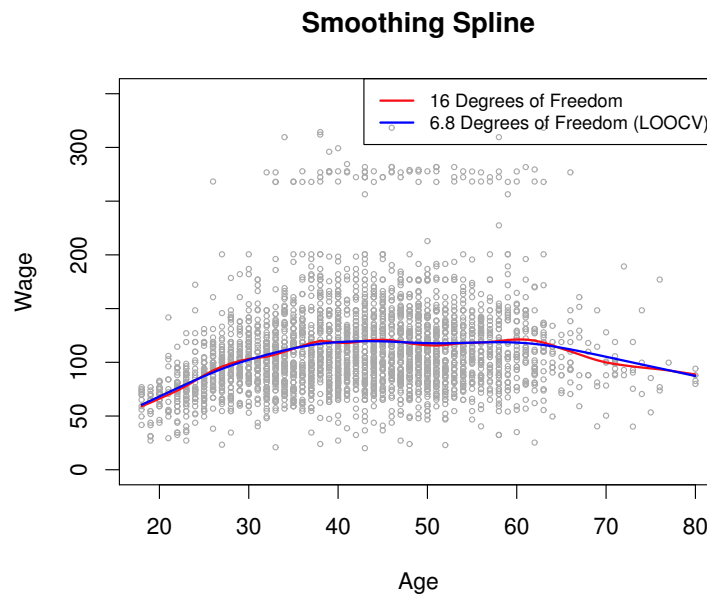
9.26

- We can specify df rather than λ !
In R: `smooth.spline(age, wage, df = 10)`
- The leave-one-out (LOO) cross-validated error is given by

$$RSS_{cv}(\lambda) = \sum_{i=1}^n \left(y_i - \hat{g}_\lambda^{(-i)}(x_i) \right)^2 = \sum_{i=1}^n \left[\frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{\mathbf{S}_\lambda\}_{ii}} \right]^2.$$

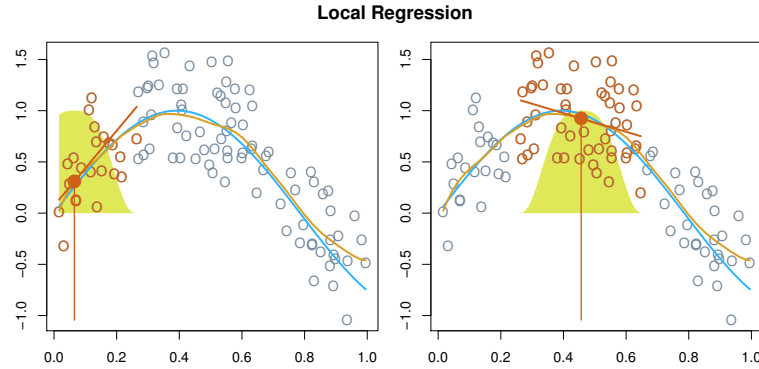
In R: `smooth.spline(age, wage)`

9.27



9.28

11 Local Regression



With a sliding weight function (weights change with point x_0), we fit separate linear fits over the range of X by weighted least squares, according to the following algorithm.

9.29

Local Regression at x_0

- Gather the fraction $s = k/n$ of training points whose x_i are closest to x_0 .
- Assign a weight $K_{i0} = K(x_i, x_0)$ to each point in this neighborhood, so that the point furthest from x_0 has weight zero, and the closest has the highest weight. All but these k nearest neighbors get weight zero.
- Fit a weighted least squares regression of Y on X , using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

$$\hat{\beta}_0, \hat{\beta}_1 = \arg \min_{\beta_0, \beta_1} \sum_{i=1}^n K_{i0} (y_i - \beta_0 - \beta_1 x_i)^2.$$

- The fitted value of Y at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

9.30

Local Regression coefficient estimates vary locally.

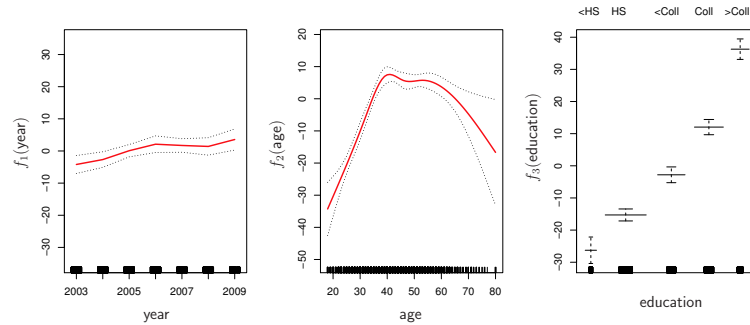
Can be estimated in R with `loess()`.

9.31

12 Generalized Additive Models

Allows for flexible nonlinearities in several variables, but retains the additive structure of linear models.

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \dots + f_p(x_{ip}) + \varepsilon_i.$$



9.32

GAM details

- Can fit a GAM simply using, e.g. natural splines:
`lm(wage ~ ns(year, df= 5) + ns(age, df = 5) + education)`
- Coefficients not that interesting; fitted functions are. The previous plot was produced using `plot.gam`.
- Can mix terms – some linear, some nonlinear – and use `anova()` to compare models.
- Can use smoothing splines or local regression as well:
`gam(wage ~ s(year, df = 5) + lo(age, span = .5) + education)`
- GAMs are additive, although low-order interactions can be included in a natural way using, e.g. bivariate smoothers or interactions of the form
`ns(age, df = 5):ns(year, df = 5).`

9.33